

pairs, called an input split in Hadoop. Input splits are computed automatically and the execution framework strives to align them to HDFS block boundaries so that each map task is associated with a single data block. In scheduling map tasks, the jobtracker tries to take advantage of data locality—if possible, map tasks are scheduled on the slave node that holds the input split, so that the mapper will be processing local data. The alignment of input splits with HDFS block boundaries simplifies task scheduling. If it is not possible to run a map task on local data, it becomes necessary to stream input key-value pairs across the network. Since large clusters are organized into racks, with far greater intra-rack bandwidth than inter-rack bandwidth, the execution framework strives to at least place map tasks on a rack which has a copy of the data block.

Although conceptually in MapReduce one can think of the mapper being applied to all input key-value pairs and the reducer being applied to all values associated with the same key, actual job execution is a bit more complex. In Hadoop, mappers are Java objects with a `MAP` method (among others). A mapper object is instantiated for every map task by the tasktracker. The life-cycle of this object begins with instantiation, where a hook is provided in the API to run programmer-specified code. This means that mappers can read in “side data”, providing an opportunity to load state, static data sources, dictionaries, etc. After initialization, the `MAP` method is called (by the execution framework) on all key-value pairs in the input split. Since these method calls occur in the context of the same Java object, it is possible to preserve state across multiple input key-value pairs within the same map task—this is an important property to exploit in the design of MapReduce algorithms, as we will see in the next chapter. After all key-value pairs in the input split have been processed, the mapper object provides an opportunity to run programmer-specified termination code. This, too, will be important in the design of MapReduce algorithms.

The actual execution of reducers is similar to that of the mappers. Each reducer object is instantiated for every reduce task. The Hadoop API provides hooks for programmer-specified initialization and termination code. After initialization, for each intermediate key in the partition (defined by the partitioner), the execution framework repeatedly calls the `REDUCE` method with an intermediate key and an iterator over all values associated with that key. The programming model also guarantees that intermediate keys will be presented to the `REDUCE` method in sorted order. Since this occurs in the context of a single object, it is possible to preserve state across multiple intermediate keys (and associated values) within a single reduce task. Once again, this property is critical in the design of MapReduce algorithms and will be discussed in the next chapter.

2.7 SUMMARY

This chapter provides a basic overview of the MapReduce programming model, starting with its roots in functional programming and continuing with a description of mappers, reducers, partitioners, and combiners. Significant attention is also given to the underlying distributed file system, which is a tightly-integrated component of the MapReduce environment. Given this basic understanding, we now turn our attention to the design of MapReduce algorithms.

CHAPTER 3

MapReduce Algorithm Design

A large part of the power of MapReduce comes from its simplicity: in addition to preparing the input data, the programmer needs only to implement the mapper, the reducer, and optionally, the combiner and the partitioner. All other aspects of execution are handled transparently by the execution framework—on clusters ranging from a single node to a few thousand nodes, over datasets ranging from gigabytes to petabytes. However, this also means that any conceivable algorithm that a programmer wishes to develop must be expressed in terms of a small number of rigidly-defined components that must fit together in very specific ways. It may not appear obvious how a multitude of algorithms can be recast into this programming model. The purpose of this chapter is to provide, primarily through examples, a guide to MapReduce algorithm design. These examples illustrate what can be thought of as “design patterns” for MapReduce, which instantiate arrangements of components and specific techniques designed to handle frequently-encountered situations across a variety of problem domains. Two of these design patterns are used in the scalable inverted indexing algorithm we’ll present later in Chapter 4; concepts presented here will show up again in Chapter 5 (graph processing) and Chapter 6 (expectation-maximization algorithms).

Synchronization is perhaps the most tricky aspect of designing MapReduce algorithms (or for that matter, parallel and distributed algorithms in general). Other than embarrassingly-parallel problems, processes running on separate nodes in a cluster must, at some point in time, come together—for example, to distribute partial results from nodes that produced them to the nodes that will consume them. Within a single MapReduce job, there is only one opportunity for cluster-wide synchronization—during the shuffle and sort stage where intermediate key-value pairs are copied from the mappers to the reducers and grouped by key. Beyond that, mappers and reducers run in isolation without any mechanisms for direct communication. Furthermore, the programmer has little control over many aspects of execution, for example:

- *Where* a mapper or reducer runs (i.e., on which node in the cluster).
- *When* a mapper or reducer begins or finishes.
- *Which* input key-value pairs are processed by a specific mapper.
- *Which* intermediate key-value pairs are processed by a specific reducer.

Nevertheless, the programmer does have a number of techniques for controlling execution and managing the flow of data in MapReduce. In summary, they are:

1. The ability to construct complex data structures as keys and values to store and communicate partial results.
2. The ability to execute user-specified initialization code at the beginning of a map or reduce task, and the ability to execute user-specified termination code at the end of a map or reduce task.
3. The ability to preserve state in both mappers and reducers across multiple input or intermediate keys.
4. The ability to control the sort order of intermediate keys, and therefore the order in which a reducer will encounter particular keys.
5. The ability to control the partitioning of the key space, and therefore the set of keys that will be encountered by a particular reducer.

It is important to realize that many algorithms cannot be easily expressed as a single MapReduce job. One must often decompose complex algorithms into a sequence of jobs, which requires orchestrating data so that the output of one job becomes the input to the next. Many algorithms are iterative in nature, requiring repeated execution until some convergence criteria—graph algorithms in Chapter 5 and expectation-maximization algorithms in Chapter 6 behave in exactly this way. Often, the convergence check itself cannot be easily expressed in MapReduce. The standard solution is an external (non-MapReduce) program that serves as a “driver” to coordinate MapReduce iterations.

This chapter explains how various techniques to control code execution and data flow can be applied to design algorithms in MapReduce. The focus is both on scalability—ensuring that there are no inherent bottlenecks as algorithms are applied to increasingly larger datasets—and efficiency—ensuring that algorithms do not needlessly consume resources and thereby reducing the cost of parallelization. The gold standard, of course, is linear scalability: an algorithm running on twice the amount of data should take only twice as long. Similarly, an algorithm running on twice the number of nodes should only take half as long.

The chapter is organized as follows:

- Section 3.1 introduces the important concept of local aggregation in MapReduce and strategies for designing efficient algorithms that minimize the amount of partial results that need to be copied across the network. The proper use of combiners is discussed in detail, as well as the “in-mapper combining” design pattern.
- Section 3.2 uses the example of building word co-occurrence matrices on large text corpora to illustrate two common design patterns, which we dub “pairs” and “stripes”. These two approaches are useful in a large class of problems that require keeping track of joint events across a large number of observations.

- Section 3.3 shows how co-occurrence counts can be converted into relative frequencies using a pattern known as “order inversion”. The sequencing of computations in the reducer can be recast as a sorting problem, where pieces of intermediate data are sorted into exactly the order that is required to carry out a series of computations. Often, a reducer needs to compute an aggregate statistic on a set of elements before individual elements can be processed. Normally, this would require two passes over the data, but with the “order inversion” design pattern, the aggregate statistic can be computed in the reducer before the individual elements are encountered. This may seem counter-intuitive: how can we compute an aggregate statistic on a set of elements before encountering elements of that set? As it turns out, clever sorting of special key-value pairs enables exactly this.
- Section 3.4 provides a general solution to secondary sorting, which is the problem of sorting values associated with a key in the reduce phase. We call this technique “value-to-key conversion”.
- Section 3.5 covers the topic of performing joins on relational datasets and presents three different approaches: *reduce-side*, *map-side*, and *memory-backed* joins.

3.1 LOCAL AGGREGATION

In the context of data-intensive distributed processing, the single most important aspect of synchronization is the exchange of intermediate results, from the processes that produced them to the processes that will ultimately consume them. In a cluster environment, with the exception of embarrassingly-parallel problems, this necessarily involves transferring data over the network. Furthermore, in Hadoop, intermediate results are written to local disk before being sent over the network. Since network and disk latencies are relatively expensive compared to other operations, reductions in the amount of intermediate data translate into increases in algorithmic efficiency. In MapReduce, local aggregation of intermediate results is one of the keys to efficient algorithms. Through use of the combiner and by taking advantage of the ability to preserve state across multiple inputs, it is often possible to substantially reduce both the number and size of key-value pairs that need to be shuffled from the mappers to the reducers.

3.1.1 COMBINERS AND IN-MAPPER COMBINING

We illustrate various techniques for local aggregation using the simple word count example presented in Section 2.2. For convenience, Figure 3.1 repeats the pseudo-code of the basic algorithm, which is quite simple: the mapper emits an intermediate key-value pair for each term observed, with the term itself as the key and a value of one; reducers sum up the partial counts to arrive at the final count.

```

1: class MAPPER
2:   method MAP(docid  $a$ , doc  $d$ )
3:     for all term  $t \in \text{doc } d$  do
4:       EMIT(term  $t$ , count 1)

1: class REDUCER
2:   method REDUCE(term  $t$ , counts  $[c_1, c_2, \dots]$ )
3:      $sum \leftarrow 0$ 
4:     for all count  $c \in \text{counts } [c_1, c_2, \dots]$  do
5:        $sum \leftarrow sum + c$ 
6:     EMIT(term  $t$ , count  $sum$ )

```

Figure 3.1: Pseudo-code for the basic word count algorithm in MapReduce (repeated from Figure 2.3).

The first technique for local aggregation is the combiner, already discussed in Section 2.4. Combiners provide a general mechanism within the MapReduce framework to reduce the amount of intermediate data generated by the mappers—recall that they can be understood as “mini-reducers” that process the output of mappers. In this example, the combiners aggregate term counts across the documents processed by each map task. This results in a reduction in the number of intermediate key-value pairs that need to be shuffled across the network—from the order of *total* number of terms in the collection to the order of the number of *unique* terms in the collection.¹

An improvement on the basic algorithm is shown in Figure 3.2 (the mapper is modified but the reducer remains the same as in Figure 3.1 and therefore is not repeated). An associative array (i.e., Map in Java) is introduced inside the mapper to tally up term counts within a single document: instead of emitting a key-value pair for each term in the document, this version emits a key-value pair for each *unique* term in the document. Given that some words appear frequently within a document (for example, a document about dogs is likely to have many occurrences of the word “dog”), this can yield substantial savings in the number of intermediate key-value pairs emitted, especially for long documents.

¹More precisely, if the combiners take advantage of all opportunities for local aggregation, the algorithm would generate at most $m \times V$ intermediate key-value pairs, where m is the number of mappers and V is the vocabulary size (number of unique terms in the collection), since every term could have been observed in every mapper. However, there are two additional factors to consider. Due to the Zipfian nature of term distributions, most terms will not be observed by most mappers (for example, terms that occur only once will by definition only be observed by one mapper). On the other hand, combiners in Hadoop are treated as *optional* optimizations, so there is no guarantee that the execution framework will take advantage of all opportunities for partial aggregation.

```

1: class MAPPER
2:   method MAP(docid  $a$ , doc  $d$ )
3:      $H \leftarrow \text{new ASSOCIATIVEARRAY}$ 
4:     for all term  $t \in \text{doc } d$  do
5:        $H\{t\} \leftarrow H\{t\} + 1$  ▷ Tally counts for entire document
6:     for all term  $t \in H$  do
7:       EMIT(term  $t$ , count  $H\{t\}$ )

```

Figure 3.2: Pseudo-code for the improved MapReduce word count algorithm that uses an associative array to aggregate term counts on a per-document basis. Reducer is the same as in Figure 3.1.

This basic idea can be taken one step further, as illustrated in the variant of the word count algorithm in Figure 3.3 (once again, only the mapper is modified). The workings of this algorithm critically depends on the details of how map and reduce tasks in Hadoop are executed, discussed in Section 2.6. Recall, a (Java) mapper object is created for each map task, which is responsible for processing a block of input key-value pairs. Prior to processing any input key-value pairs, the mapper’s INITIALIZE method is called, which is an API hook for user-specified code. In this case, we initialize an associative array for holding term counts. Since it is possible to preserve state across multiple calls of the MAP method (for each input key-value pair), we can continue to accumulate partial term counts in the associative array *across* multiple documents, and emit key-value pairs only when the mapper has processed all documents. That is, emission of intermediate data is deferred until the CLOSE method in the pseudo-code. Recall that this API hook provides an opportunity to execute user-specified code *after* the MAP method has been applied to all input key-value pairs of the input data split to which the map task was assigned.

With this technique, we are in essence incorporating combiner functionality directly inside the mapper. There is no need to run a separate combiner, since all opportunities for local aggregation are already exploited.² This is a sufficiently common design pattern in MapReduce that it’s worth giving it a name, “in-mapper combining”, so that we can refer to the pattern more conveniently throughout the book. We’ll see later on how this pattern can be applied to a variety of problems. There are two main advantages to using this design pattern:

First, it provides control over when local aggregation occurs and how it exactly takes place. In contrast, the semantics of the combiner is underspecified in MapReduce.

²Leaving aside the minor complication that in Hadoop, combiners can be run in the reduce phase also (when merging intermediate key-value pairs from different map tasks). However, in practice it makes almost no difference either way.

```

1: class MAPPER
2:   method INITIALIZE
3:      $H \leftarrow \text{new ASSOCIATIVEARRAY}$ 
4:   method MAP(docid  $a$ , doc  $d$ )
5:     for all term  $t \in \text{doc } d$  do
6:        $H\{t\} \leftarrow H\{t\} + 1$  ▷ Tally counts across documents
7:   method CLOSE
8:     for all term  $t \in H$  do
9:       EMIT(term  $t$ , count  $H\{t\}$ )

```

Figure 3.3: Pseudo-code for the improved MapReduce word count algorithm that demonstrates the “in-mapper combining” design pattern. Reducer is the same as in Figure 3.1.

For example, Hadoop makes no guarantees on how many times the combiner is applied, or that it is even applied at all. The combiner is provided as a semantics-preserving optimization to the execution framework, which has the *option* of using it, perhaps multiple times, or not at all (or even in the reduce phase). In some cases (although not in this particular example), such indeterminism is unacceptable, which is exactly why programmers often choose to perform their own local aggregation in the mappers.

Second, in-mapper combining will typically be more efficient than using actual combiners. One reason for this is the additional overhead associated with actually materializing the key-value pairs. Combiners reduce the amount of intermediate data that is shuffled across the network, but don’t actually reduce the number of key-value pairs that are emitted by the mappers in the first place. With the algorithm in Figure 3.2, intermediate key-value pairs are still generated on a per-document basis, only to be “compacted” by the combiners. This process involves unnecessary object creation and destruction (garbage collection takes time), and furthermore, object serialization and deserialization (when intermediate key-value pairs fill the in-memory buffer holding map outputs and need to be temporarily spilled to disk). In contrast, with in-mapper combining, the mappers will generate only those key-value pairs that need to be shuffled across the network to the reducers.

There are, however, drawbacks to the in-mapper combining pattern. First, it breaks the functional programming underpinnings of MapReduce, since state is being preserved across multiple input key-value pairs. Ultimately, this isn’t a big deal, since pragmatic concerns for efficiency often trump theoretical “purity”, but there are practical consequences as well. Preserving state across multiple input instances means that algorithmic behavior may depend on the order in which input key-value pairs are encountered. This creates the potential for ordering-dependent bugs, which are difficult to debug on large datasets in the general case (although the correctness of in-mapper

combining for word count is easy to demonstrate). Second, there is a fundamental scalability bottleneck associated with the in-mapper combining pattern. It critically depends on having sufficient memory to store intermediate results until the mapper has completely processed all key-value pairs in an input split. In the word count example, the memory footprint is bound by the vocabulary size, since it is theoretically possible that a mapper encounters every term in the collection. Heap’s Law, a well-known result in information retrieval, accurately models the growth of vocabulary size as a function of the collection size—the somewhat surprising fact is that the vocabulary size never stops growing.³ Therefore, the algorithm in Figure 3.3 will scale only up to a point, beyond which the associative array holding the partial term counts will no longer fit in memory.⁴

One common solution to limiting memory usage when using the in-mapper combining technique is to “block” input key-value pairs and “flush” in-memory data structures periodically. The idea is simple: instead of emitting intermediate data only after *every* key-value pair has been processed, emit partial results after processing every n key-value pairs. This is straightforwardly implemented with a counter variable that keeps track of the number of input key-value pairs that have been processed. As an alternative, the mapper could keep track of its own memory footprint and flush intermediate key-value pairs once memory usage has crossed a certain threshold. In both approaches, either the block size or the memory usage threshold needs to be determined empirically: with too large a value, the mapper may run out of memory, but with too small a value, opportunities for local aggregation may be lost. Furthermore, in Hadoop physical memory is split between multiple tasks that may be running on a node concurrently; these tasks are all competing for finite resources, but since the tasks are not aware of each other, it is difficult to coordinate resource consumption effectively. In practice, however, one often encounters diminishing returns in performance gains with increasing buffer sizes, such that it is not worth the effort to search for an *optimal* buffer size (personal communication, Jeff Dean).

In MapReduce algorithms, the extent to which efficiency can be increased through local aggregation depends on the size of the intermediate key space, the distribution of keys themselves, and the number of key-value pairs that are emitted by each individual map task. Opportunities for aggregation, after all, come from having multiple values associated with the same key (whether one uses combiners or employs the in-mapper combining pattern). In the word count example, local aggregation is effective because

³In more detail, Heap’s Law relates the vocabulary size V to the collection size as follows: $V = kT^b$, where T is the number of tokens in the collection. Typical values of the parameters k and b are: $30 \leq k \leq 100$ and $b \sim 0.5$ ([101], p. 81).

⁴A few more details: note what matters is that the partial term counts encountered within particular *input split* fits into memory. However, as collection sizes increase, one will often want to increase the input split size to limit the growth of the number of map tasks (in order to reduce the number of distinct copy operations necessary to shuffle intermediate data over the network).

many words are encountered multiple times within a map task. Local aggregation is also an effective technique for dealing with reduce stragglers (see Section 2.3) that result from a highly-skewed (e.g., Zipfian) distribution of values associated with intermediate keys. In our word count example, we do not filter frequently-occurring words: therefore, without local aggregation, the reducer that’s responsible for computing the count of ‘the’ will have a lot more work to do than the typical reducer, and therefore will likely be a straggler. With local aggregation (either combiners or in-mapper combining), we substantially reduce the number of values associated with frequently-occurring terms, which alleviates the reduce straggler problem.

3.1.2 ALGORITHMIC CORRECTNESS WITH LOCAL AGGREGATION

Although use of combiners can yield dramatic reductions in algorithm running time, care must be taken in applying them. Since combiners in Hadoop are viewed as optional optimizations, the correctness of the algorithm cannot depend on computations performed by the combiner or depend on them even being run at all. In any MapReduce program, the reducer input key-value type must match the mapper output key-value type: this implies that the combiner input *and* output key-value types must match the mapper output key-value type (which is the same as the reducer input key-value type). In cases where the reduce computation is both commutative and associative, the reducer can also be used (unmodified) as the combiner (as is the case with the word count example). In the general case, however, combiners and reducers are not interchangeable.

Consider a simple example: we have a large dataset where input keys are strings and input values are integers, and we wish to compute the mean of all integers associated with the same key (rounded to the nearest integer). A real-world example might be a large user log from a popular website, where keys represent user ids and values represent some measure of activity such as elapsed time for a particular session—the task would correspond to computing the mean session length on a per-user basis, which would be useful for understanding user demographics. Figure 3.4 shows the pseudo-code of a simple algorithm for accomplishing this task that does not involve combiners. We use an identity mapper, which simply passes all input key-value pairs to the reducers (appropriately grouped and sorted). The reducer keeps track of the running sum and the number of integers encountered. This information is used to compute the mean once all values are processed. The mean is then emitted as the output value in the reducer (with the input string as the key).

This algorithm will indeed work, but suffers from the same drawbacks as the basic word count algorithm in Figure 3.1: it requires shuffling all key-value pairs from mappers to reducers across the network, which is highly inefficient. Unlike in the word count example, the reducer cannot be used as a combiner in this case. Consider what would happen if we did: the combiner would compute the mean of an arbitrary subset

```

1: class MAPPER
2:   method MAP(string  $t$ , integer  $r$ )
3:     EMIT(string  $t$ , integer  $r$ )
1: class REDUCER
2:   method REDUCE(string  $t$ , integers  $[r_1, r_2, \dots]$ )
3:      $sum \leftarrow 0$ 
4:      $cnt \leftarrow 0$ 
5:     for all integer  $r \in$  integers  $[r_1, r_2, \dots]$  do
6:        $sum \leftarrow sum + r$ 
7:        $cnt \leftarrow cnt + 1$ 
8:      $r_{avg} \leftarrow sum / cnt$ 
9:     EMIT(string  $t$ , integer  $r_{avg}$ )

```

Figure 3.4: Pseudo-code for the basic MapReduce algorithm that computes the mean of values associated with the same key.

of values associated with the same key, and the reducer would compute the mean of those values. As a concrete example, we know that:

$$\text{MEAN}(1, 2, 3, 4, 5) \neq \text{MEAN}(\text{MEAN}(1, 2), \text{MEAN}(3, 4, 5))$$

In general, the mean of means of arbitrary subsets of a set of numbers is not the same as the mean of the set of numbers. Therefore, this approach would not produce the correct result.⁵

So how might we properly take advantage of combiners? An attempt is shown in Figure 3.5. The mapper remains the same, but we have added a combiner that partially aggregates results by computing the numeric components necessary to arrive at the mean. The combiner receives each string and the associated list of integer values, from which it computes the sum of those values and the number of integers encountered (i.e., the count). The sum and count are packaged into a pair, and emitted as the output of the combiner, with the same string as the key. In the reducer, pairs of partial sums and counts can be aggregated to arrive at the mean. Up until now, all keys and values in our algorithms have been primitives (string, integers, etc.). However, there are no prohibitions in MapReduce for more complex types,⁶ and, in fact, this represents a key technique in MapReduce algorithm design that we introduced at the beginning of this

⁵There is, however, one special case in which using reducers as combiners *would* produce the correct result: if each combiner computed the mean of equal-size subsets of the values. However, since such fine-grained control over the combiners is impossible in MapReduce, such a scenario is highly unlikely.

⁶In Hadoop, either custom types or types defined using a library such as Protocol Buffers, Thrift, or Avro.

```

1: class MAPPER
2:   method MAP(string  $t$ , integer  $r$ )
3:     EMIT(string  $t$ , integer  $r$ )

1: class COMBINER
2:   method COMBINE(string  $t$ , integers  $[r_1, r_2, \dots]$ )
3:      $sum \leftarrow 0$ 
4:      $cnt \leftarrow 0$ 
5:     for all integer  $r \in$  integers  $[r_1, r_2, \dots]$  do
6:        $sum \leftarrow sum + r$ 
7:        $cnt \leftarrow cnt + 1$ 
8:     EMIT(string  $t$ , pair  $(sum, cnt)$ ) ▷ Separate sum and count

1: class REDUCER
2:   method REDUCE(string  $t$ , pairs  $[(s_1, c_1), (s_2, c_2) \dots]$ )
3:      $sum \leftarrow 0$ 
4:      $cnt \leftarrow 0$ 
5:     for all pair  $(s, c) \in$  pairs  $[(s_1, c_1), (s_2, c_2) \dots]$  do
6:        $sum \leftarrow sum + s$ 
7:        $cnt \leftarrow cnt + c$ 
8:      $r_{avg} \leftarrow sum / cnt$ 
9:     EMIT(string  $t$ , integer  $r_{avg}$ )

```

Figure 3.5: Pseudo-code for an incorrect first attempt at introducing combiners to compute the mean of values associated with each key. The mismatch between combiner input and output key-value types violates the MapReduce programming model.

chapter. We will frequently encounter complex keys and values throughout the rest of this book.

Unfortunately, this algorithm will not work. Recall that combiners must have the same input and output key-value type, which also must be the same as the mapper output type and the reducer input type. This is clearly not the case. To understand why this restriction is necessary in the programming model, remember that combiners are optimizations that cannot change the correctness of the algorithm. So let us remove the combiner and see what happens: the output value type of the mapper is integer, so the reducer expects to receive a list of integers as values. But the reducer actually expects a list of pairs! The correctness of the algorithm is contingent on the combiner running on the output of the mappers, and more specifically, that the combiner is run exactly once. Recall from our previous discussion that Hadoop makes no guarantees on

```

1: class MAPPER
2:   method MAP(string  $t$ , integer  $r$ )
3:     EMIT(string  $t$ , pair  $(r, 1)$ )

1: class COMBINER
2:   method COMBINE(string  $t$ , pairs  $[(s_1, c_1), (s_2, c_2) \dots]$ )
3:      $sum \leftarrow 0$ 
4:      $cnt \leftarrow 0$ 
5:     for all pair  $(s, c) \in$  pairs  $[(s_1, c_1), (s_2, c_2) \dots]$  do
6:        $sum \leftarrow sum + s$ 
7:        $cnt \leftarrow cnt + c$ 
8:     EMIT(string  $t$ , pair  $(sum, cnt)$ )

1: class REDUCER
2:   method REDUCE(string  $t$ , pairs  $[(s_1, c_1), (s_2, c_2) \dots]$ )
3:      $sum \leftarrow 0$ 
4:      $cnt \leftarrow 0$ 
5:     for all pair  $(s, c) \in$  pairs  $[(s_1, c_1), (s_2, c_2) \dots]$  do
6:        $sum \leftarrow sum + s$ 
7:        $cnt \leftarrow cnt + c$ 
8:      $r_{avg} \leftarrow sum / cnt$ 
9:     EMIT(string  $t$ , integer  $r_{avg}$ )

```

Figure 3.6: Pseudo-code for a MapReduce algorithm that computes the mean of values associated with each key. This algorithm correctly takes advantage of combiners.

how many times combiners are called; it could be zero, one, or multiple times. This violates the MapReduce programming model.

Another stab at the algorithm is shown in Figure 3.6, and this time, the algorithm is correct. In the mapper we emit as the value a pair consisting of the integer and one—this corresponds to a partial count over one instance. The combiner separately aggregates the partial sums and the partial counts (as before), and emits pairs with updated sums and counts. The reducer is similar to the combiner, except that the mean is computed at the end. In essence, this algorithm transforms a non-associative operation (mean of numbers) into an associative operation (element-wise sum of a pair of numbers, with an additional division at the very end).

Let us verify the correctness of this algorithm by repeating the previous exercise: What would happen if no combiners were run? With no combiners, the mappers would send pairs (as values) directly to the reducers. There would be as many intermediate pairs as there were input key-value pairs, and each of those would consist of an integer

```

1: class MAPPER
2:   method INITIALIZE
3:      $S \leftarrow \text{new ASSOCIATIVEARRAY}$ 
4:      $C \leftarrow \text{new ASSOCIATIVEARRAY}$ 
5:   method MAP(string  $t$ , integer  $r$ )
6:      $S\{t\} \leftarrow S\{t\} + r$ 
7:      $C\{t\} \leftarrow C\{t\} + 1$ 
8:   method CLOSE
9:     for all term  $t \in S$  do
10:      EMIT(term  $t$ , pair ( $S\{t\}$ ,  $C\{t\}$ ))

```

Figure 3.7: Pseudo-code for a MapReduce algorithm that computes the mean of values associated with each key, illustrating the in-mapper combining design pattern. Only the mapper is shown here; the reducer is the same as in Figure 3.6

and one. The reducer would still arrive at the correct sum and count, and hence the mean would be correct. Now add in the combiners: the algorithm would remain correct, no matter how many times they run, since the combiners merely aggregate partial sums and counts to pass along to the reducers. Note that although the output key-value type of the combiner must be the same as the input key-value type of the reducer, the reducer can emit final key-value pairs of a different type.

Finally, in Figure 3.7, we present an even more efficient algorithm that exploits the in-mapper combining pattern. Inside the mapper, the partial sums and counts associated with each string are held in memory across input key-value pairs. Intermediate key-value pairs are emitted only after the entire input split has been processed; similar to before, the value is a pair consisting of the sum and count. The reducer is exactly the same as in Figure 3.6. Moving partial aggregation from the combiner directly into the mapper is subjected to all the tradeoffs and caveats discussed earlier this section, but in this case the memory footprint of the data structures for holding intermediate data is likely to be modest, making this variant algorithm an attractive option.

3.2 PAIRS AND STRIPES

One common approach for synchronization in MapReduce is to construct complex keys and values in such a way that data necessary for a computation are naturally brought together by the execution framework. We first touched on this technique in the previous section, in the context of “packaging” partial sums and counts in a complex value (i.e., pair) that is passed from mapper to combiner to reducer. Building on previously